Core Java Interview Questions

Structured notes for revision and interview preparation

1. Why Java is not 100 percent Object oriented?

Answer: Java is not 100 percent object-oriented because it supports primitive data types.

Key points:

- Primitive types include boolean, byte, char, int, float, double, long, and short.
- Primitive values are not objects, which is why Java is not treated as a pure object-oriented language.
- Wrapper classes such as Integer, Boolean, Character, Float, and Double wrap primitive values into objects when object behavior is needed.

2. Why pointers are not used in Java?

Answer: Java does not expose pointers because they are unsafe, complex, and allow direct memory access.

Key points:

- Pointers can make programs more complex, while Java focuses on simplicity.
- The JVM is responsible for memory allocation and garbage collection.
- Java uses references, but it does not allow pointer arithmetic or direct memory manipulation by the programmer.

3. What is JIT compiler?

Answer: JIT stands for Just-In-Time compiler. It is a JVM component that converts bytecode into machine code during program execution.

Key points:

- Java source code is compiled by javac into bytecode.
- The JVM can interpret bytecode.
- The JIT compiler improves performance by compiling frequently executed bytecode into native machine code.

JIT Compiler Flow

4. Why String is immutable in Java?

Answer: String is immutable so that once a String object is created, its value cannot be changed.

Key points:

- The String pool requires immutability because multiple references can point to the same String object.
- If shared String values were mutable, a change from one reference could affect other parts of the application.
- String immutability improves security for values used in file paths, network connections, class loading, and database connections.

5. What is marker interface in Java?

Answer: A marker interface is an empty interface that has no data members and no methods.

Key points:

- It is used to mark or tag a class so the JVM or framework can treat it specially.
- Serializable and Cloneable are common examples.
- A class implements a marker interface to declare a capability without implementing interface methods.

6. Can you override a private or static method in Java?

Answer: No. A private method cannot be overridden, and a static method is hidden, not overridden.

Key points:

- A private method is not accessible in the subclass, so the subclass cannot override it.
- If a subclass declares a private method with the same name, it is a separate method.
- If a subclass declares a static method with the same signature, it hides the superclass static method. This is called method hiding.

7. Does finally always execute in Java?

Answer: Usually yes, but not in every situation.

Key points:

- finally may not execute if System.exit() is called.
- finally may not execute if the JVM or system crashes.

8. What methods does Object class have?

Answer: java.lang.Object is the parent class of all Java classes and provides common methods.

Method	Purpose
clone()	Creates and returns a copy of the object.
equals(Object obj)	Checks whether another object is equal to this object.
finalize()	Called by the garbage collector before reclaiming the object.
getClass()	Returns the runtime class of the object.

Method	Purpose
hashCode()	Returns the hash code value for the object.
toString()	Returns the string representation of the object.
notify()	Wakes one thread waiting on this object's monitor.
notifyAll()	Wakes all threads waiting on this object's monitor.
wait()	Causes the current thread to wait.
wait(long timeout)	Waits for the specified timeout.
wait(long timeout, int nanos)	Waits for the specified timeout and nanoseconds.

9. How can you make a class immutable?

Answer: An immutable class is designed so that its state cannot be changed after object creation.

Key points:

- Declare the class as final so it cannot be extended.
- Make all fields private.
- Do not provide setter methods.
- Make mutable fields final where possible.
- Initialize fields through a constructor.
- Use deep copy while assigning mutable objects.
- Return copies from getters instead of returning mutable object references directly.

10. What is singleton class and how can we make a singleton class in Java?

Answer: A singleton class allows only one instance to be created in one JVM.

Key points:

- Keep the constructor private.
- Store the single instance in a static field.
- Expose the instance using a static getInstance() method.
- In multithreaded code, a simple lazy singleton can break if two threads create the instance at the same time.
- Use synchronization or double-checked locking with volatile to make lazy singleton creation thread-safe.

Code example:

```
private static volatile Singleton instance;

private Singleton() {
}

public static Singleton getInstance() {
    if (instance == null) {
        synchronized (Singleton.class) {
            if (instance == null) {
                instance = new Singleton();
            }
        }
    }
    return instance;
}
```

11. equals() and hashCode() contract

Answer: If two objects are equal according to equals(), they must have the same hashCode().

Key points:

- If a.equals(b) is true, then a.hashCode() and b.hashCode() must be the same.
- If two objects have the same hash code, they are not necessarily equal. That situation is called collision.
- During one execution of a Java application, hashCode() should consistently return the same integer for the same object, unless fields used in equality change.

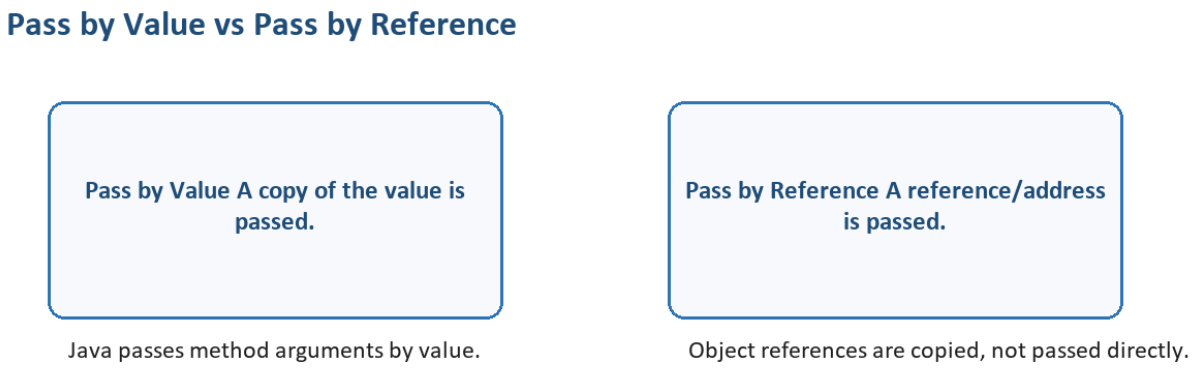
Situation	Rule
equals() returns true	hashCode() must be the same.
hashCode() is the same	Objects may or may not be equal.
Same object, same execution	hashCode() should be consistent.

12. Java is pass by value or pass by reference?

Answer: Java is pass by value.

Key points:

- For primitive values, Java passes a copy of the value.
- For objects, Java passes a copy of the reference value.
- The method receives a copied reference, so it can modify the object's internal state, but reassigning the reference inside the method does not change the caller's reference.



13. Comparable and Comparator implementation

Answer: Comparable and Comparator are used to define sorting logic for objects.

Key points:

- Comparable defines natural ordering by implementing compareTo().
- Comparator defines custom ordering by implementing compare().
- compare() or compareTo() returns a negative value, positive value, or zero based on comparison result.

Code example:

```
class NameComparator implements Comparator<Employee> {
    public int compare(Employee e1, Employee e2) {
        return e1.getName().compareTo(e2.getName());
    }
}
```

14. Why Comparable and Comparator is needed?

Answer: They are needed when objects must be sorted by natural order or by different custom criteria.

Key points:

- Comparable is useful when a class has one natural sorting order.
- Comparator is useful when the same class needs multiple sorting orders, such as name, age, or address.
- Comparator keeps sorting logic separate from the model class.

15. Comparable vs Comparator

Answer: Comparable provides natural ordering, while Comparator provides custom ordering.

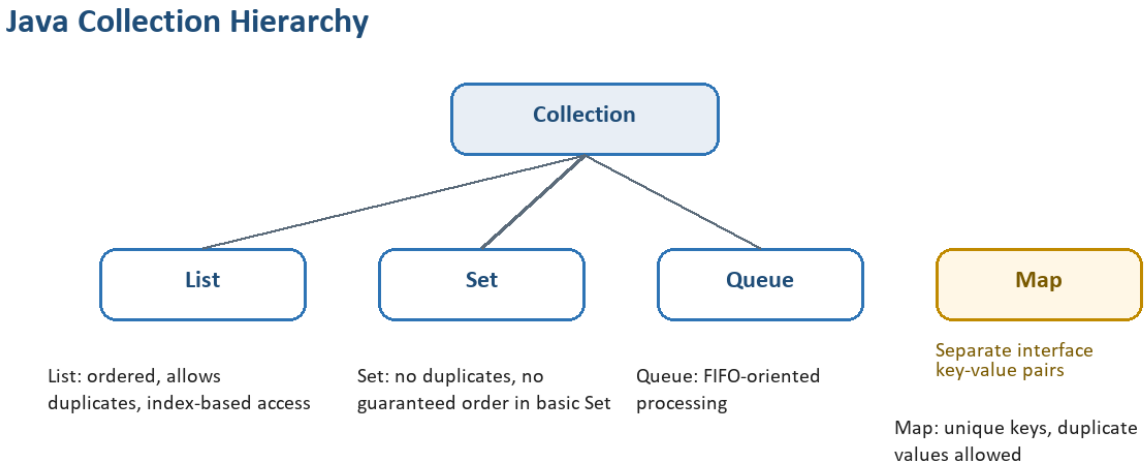
Point	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(Object o)	compare(Object o1, Object o2)
Sorting type	Natural ordering	Custom ordering
Class modification	Implemented by the class itself	Can be written outside the class
Use case	Single default sort	Multiple sorting strategies

16. Explain collection hierarchy

Answer: The Collection interface is the root of most Java collection types. Map is part of the Collections framework but does not extend Collection.

Key points:

- List contains ordered elements and can include duplicates.
- Set does not contain duplicates.
- Queue follows queue-based processing, commonly FIFO.
- Map stores key-value pairs and allows unique keys.



17. Explain List Interface

Answer: List is an ordered collection that allows duplicate elements and supports index-based access.

Implementation	Key points
ArrayList	Dynamic resizing, non-synchronized, good for random access.
LinkedList	Implements List and Deque, maintains insertion order, good for frequent insert/delete operations.
Vector	Synchronized, thread-safe, legacy class, doubles capacity when it grows.
Stack	Last-in-first-out behavior; elements are added and removed from the rear end.

18. Explain Set Interface

Answer: Set represents a collection that does not allow duplicate elements.

Implementation	Key points
HashSet	Uses hashing, stores unique elements, allows one null element, unordered.
LinkedHashSet	Ordered version of HashSet, preserves insertion order using a linked list.
SortedSet	Stores elements in sorted order; elements generally need Comparable or Comparator.
TreeSet	Uses a tree structure such as Red-Black Tree and stores elements in sorted ascending order.

19. Explain Queue Interface

Answer: Queue represents a collection designed for holding elements before processing.

Type	Key points
Queue	Usually follows FIFO, where elements are added at the rear and removed from the front.
PriorityQueue	Each element has priority; high-priority elements are served before low-priority elements.
Deque	Double-ended queue; elements can be added and removed from either end.
ArrayDeque	Resizable-array implementation of Deque with no fixed capacity restriction.

20. Explain Map Interface

Answer: Map stores data in key-value pairs. Keys are unique, while values may be duplicated.

Implementation	Key points
HashMap	Non-synchronized; allows one null key and multiple null values.
TreeMap	Maintains entries in ascending key order; uses Red-Black Tree; does not allow null key.
Hashtable	Synchronized; does not allow null key or null value.

21. Why Map does not extend Collection Interface?

Answer: Map does not extend Collection because it stores key-value pairs, while Collection stores individual objects.

Key points:

- Collection represents a group of objects with a defined access mechanism.
- Map represents key-value pairs.
- Collection's add(E e) method does not fit Map's put(K, V) behavior.
- Map is still considered an important part of the Java Collections framework.

22. Difference between fail-fast and fail-safe Iterator

Answer: Fail-fast iterators throw ConcurrentModificationException when the collection is structurally modified during iteration. Fail-safe iterators do not.

Point	Fail-fast Iterator	Fail-safe Iterator
Modification during iteration	Throws ConcurrentModificationException.	Does not throw exception.
How it works	Works directly on the original collection.	Usually works on a cloned copy of the collection.
Behavior	Fails immediately when modification is detected.	Allows iteration to continue safely.
Example idea	ArrayList iterator.	CopyOnWriteArrayList iterator.

23. What do you understand by Blocking Queue?

Answer: BlockingQueue is a thread-safe queue from java.util.concurrent that can block threads while inserting or removing elements.

Key points:

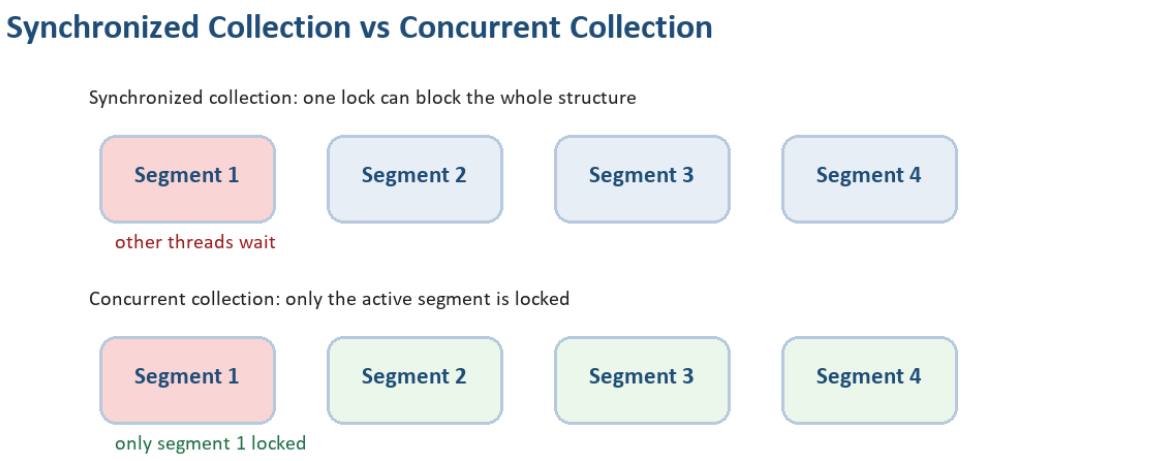
- Multiple threads can put and take elements concurrently without concurrency issues.
- If a thread tries to take an element when the queue is empty, it can be blocked until an element becomes available.
- If a thread tries to insert when the queue is full, it can be blocked until space is available.

24. Difference between synchronized collection and concurrent collection

Answer: Both provide thread safety, but concurrent collections usually provide better performance and scalability.

Key points:

- Synchronized collections use broader locking, which can make them slower.
- Concurrent collections reduce lock contention and allow better parallel access.
- ConcurrentHashMap uses segment-style locking concepts, where only a portion is locked instead of the whole structure.

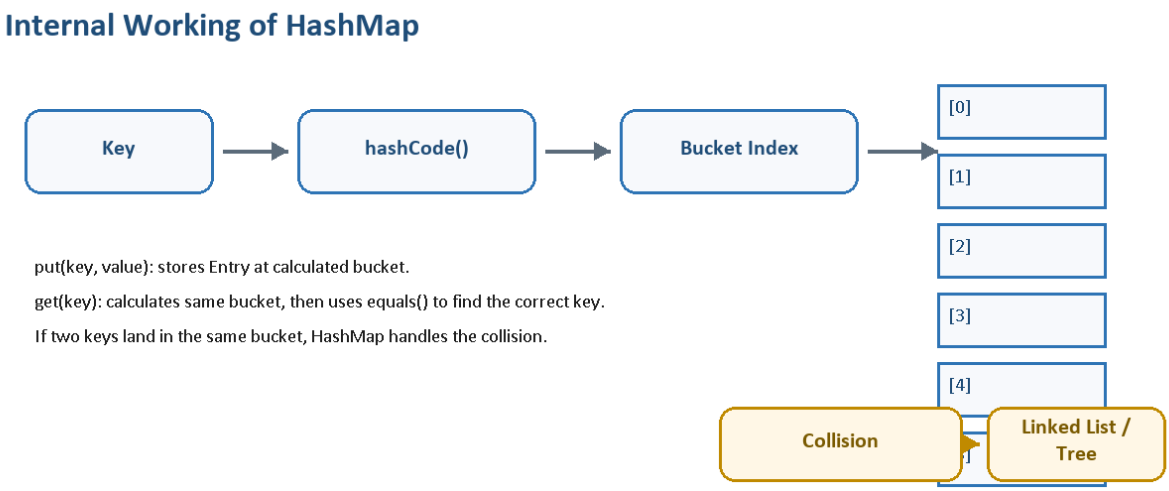


25. Internal working of HashMap

Answer: HashMap works on hashing. It stores key-value pairs in buckets based on the key's hash code.

Key points:

- put(key, value) calculates the key's hash code and uses it to identify the bucket index.
- get(key) calculates the same bucket index and retrieves the matching value.
- If two keys map to the same bucket, a collision occurs.
- During collision, entries are stored in a linked list or tree structure, and equals() is used to find the exact key.



26. Difference between array and ArrayList

Answer: Array is fixed-size, while ArrayList is dynamic-size.

Property	Array	ArrayList
Size	Static/fixed size.	Dynamic size; can resize itself.
Type support	Can store primitives and objects.	Stores objects; primitives are autoboxed.
Performance	Fast because size is fixed.	Resize operation can slow performance.
Initialization	Size must be provided.	Can be created without specifying size.
Iteration	for loop or for-each loop.	Iterator can also be used.
Length	Uses length variable.	Uses size() method.
Dimension	Can be multi-dimensional.	Always single-dimensional.
Generics	Generics are not compatible with arrays.	Supports generics.

27. Difference between ArrayList and Vector

Answer: ArrayList is non-synchronized and faster, while Vector is synchronized and slower.

Property	ArrayList	Vector
Synchronization	Not synchronized.	Synchronized.
Thread safety	Not thread-safe.	Thread-safe.
Performance	Fast because it is non-synchronized.	Slower because it is synchronized.
Legacy	Introduced in JDK 1.2; not legacy.	Legacy class.
Growth	Increases capacity by around 50%.	Doubles capacity.
Iteration	Uses Iterator.	Can use Iterator or Enumeration.

28. Difference between ArrayList and LinkedList

Answer: ArrayList uses a dynamic array, while LinkedList uses a doubly linked list.

Property	ArrayList	LinkedList
Internal implementation	Uses dynamic array.	Uses doubly linked list.
Interfaces	Implements List.	Implements List and Deque.
Add/delete	Slower for frequent middle insert/delete because elements may shift.	Faster for frequent insert/delete because links are updated.
Access/get	Faster because array supports random access.	Slower because access needs node-by-node traversal.
Reverse iteration	No descendingIterator().	Supports descendingIterator().
Initial capacity	Default initial capacity is 10 if constructor is not overloaded.	No default capacity; empty list is created.
Memory overhead	Less overhead; index holds object reference.	More overhead; each node stores next and previous addresses.

Note: Use LinkedList when frequent addition and deletion is required. Use ArrayList when search/access operations are more frequent.